

Linear Regression — Full Notes

Beginner → Intermediate → Advanced

1. Regression Models

```text

LinearRegression

Ridge

Lasso

ElasticNet

BayesianRidge

HuberRegressor

RANSACRegressor

SGDRegressor

PassiveAggressiveRegressor

TheilSenRegressor

QuantileRegressor

TweedieRegressor

PoissonRegressor

GammaRegressor

```

Beginner Level

2. What is Regression?

Regression is a **Supervised Machine Learning technique** used to predict a **continuous numerical value**.

Examples:

- House Price Prediction
- Salary Prediction
- Temperature Prediction
- Sales Prediction
- Student Marks Prediction
- Car Price Prediction

Example:

```
```text
```

Area → House Price

1000 sq ft → 200000

1500 sq ft → 300000

2000 sq ft → 400000

```
```
```

Here:

```
```text
```

Input = Area

Output = Price

```
```
```

Because Price is a continuous numerical value, this is a Regression problem.

3. What is Linear Regression?

Linear Regression finds a **linear relationship** between input X and output Y.

Basic equation:

```
```text
```

$$\hat{y} = mx + c$$

```
```
```

Where:

```
```text
```

$\hat{y}$  = Predicted value

x = Input feature

m = Slope

c = Intercept

```
```
```

Example:

```
```text
```

$$\hat{y} = 2x + 5$$

```
```
```

If:

```
```text
```

x = 10

...

Then:

```text

$$\hat{y} = 2(10) + 5$$

$$\hat{y} = 25$$

...

Prediction = 25

4. Simple Linear Regression

Simple Linear Regression is used when there is **one independent variable**.

Formula:

```text

$$\hat{y} = mx + c$$

...

Example:

```text

Hours Studied → Exam Marks

1 → 20

2 → 40

3 → 60

4 → 80

...

Relationship:

```text

Marks = 20 × Hours

...

Therefore:

```text

$$\hat{y} = 20x$$

...

If a student studies 5 hours:

```text

$$\hat{y} = 20 \times 5$$

$$\hat{y} = 100$$

...

Predicted marks = 100

---

## # 5. Independent and Dependent Variables

### ## Independent Variable

The input used to make a prediction.

Also called:

- Feature
- Predictor
- X

Example:

```text

House Area

...

Dependent Variable

The value we want to predict.

Also called:

- Target
- Output
- Y

Example:

```text

House Price

...

Relationship:

```text

Area → Price

X Y

...

6. Slope (m)

What is Slope?

Slope (m) tells us how much the dependent variable (Y) changes when the independent variable (X) changes by 1 unit.

Linear Regression-এর formula:

```
```text
```

$$m = \frac{\sum[(x_i - \bar{x})(y_i - \bar{y})]}{\sum[(x_i - \bar{x})^2]}$$

```
```
```

Where:

```
```text
```

$m$  = Slope

$x_i$  = Individual X value

$y_i$  = Individual Y value

$\bar{x}$  = Mean of X

$\bar{y}$  = Mean of Y

```
```
```

```
---
```

Example

Given:

```
```text
```

$x = [1000, 1500, 2000]$

$y = [200, 300, 400]$

```
```
```

Step 1: Calculate Mean of X

```
```text
```

$$\bar{x} = (1000 + 1500 + 2000) / 3$$

$$\bar{x} = 4500 / 3$$

$$\bar{x} = 1500$$

```
```
```

Step 2: Calculate Mean of Y

```
```text
```

$$\bar{y} = (200 + 300 + 400) / 3$$

$$\bar{y} = 900 / 3$$

$$\bar{y} = 300$$

```
```
```

Step 3: Calculate the values

| x_i | y_i | $x_i - \bar{x}$ | $y_i - \bar{y}$ | $(x_i - \bar{x})(y_i - \bar{y})$ | $(x_i - \bar{x})^2$ |
|----------------------------|----------|-----------------|-----------------|----------------------------------|-----------------------------|
| ---- | -- | ----- | ----- | ----- | ----- |
| 1000 | 200 | -500 | -100 | 50,000 | 250,000 |
| 1500 | 300 | 0 | 0 | 0 | 0 |
| 2000 | 400 | 500 | 100 | 50,000 | 250,000 |
| Σ | | | | $100,000$ | $500,000$ |

Therefore:

```
```text
```

```
m = 100000 / 500000
```

```
m = 0.2
```

```
...
```

So:

```
```text
```

```
Slope = 0.2
```

```
...
```

Interpretation

For every 1-unit increase in X, the predicted Y increases by approximately **0.2 units**, according to this model.

```
---
```

Python

```
```python
```

```
x = [1000, 1500, 2000]
```

```
y = [200, 300, 400]
```

```
x_mean = sum(x) / len(x)
```

```
y_mean = sum(y) / len(y)
```

```
numerator = sum(
```

```
(x_i - x_mean) * (y_i - y_mean)
```

```
for x_i, y_i in zip(x, y)
```

```
)
```

```
denominator = sum(
```

```
(x_i - x_mean) ** 2
```

```
for x_i in x
```

```
)
m = numerator / denominator
print(m)
...
```

Output:

```
```text  
0.2  
...
```

7. Intercept

Intercept is the predicted value of Y when X = 0.

Formula:

```
```text  

$$c = \bar{y} - m\bar{x}$$

...
```

Using the previous example:

```
```text  
x = [1000, 1500, 2000]  
y = [200, 300, 400]  
 $\bar{x} = 1500$   
 $\bar{y} = 300$   
m = 0.2  
...
```

Therefore:

```
```text  
c = 300 - (0.2 × 1500)
c = 300 - 300
c = 0
...
```

So:

```
```text  
Intercept = 0  
...
```

Therefore, the Linear Regression equation is:

```
```text
```

$$\hat{y} = mx + c$$

$$\hat{y} = 0.2x + 0$$

$$\hat{y} = 0.2x$$

```
...
```

### ### Interpretation

When:

```
```text
```

$$X = 0$$

```
...
```

the model predicts:

```
```text
```

$$Y = 0$$

```
...
```

The intercept is the point where the regression line crosses the Y-axis.

Python:

```
```python
```

$$m = 0.2$$

$$x_mean = 1500$$

$$y_mean = 300$$

$$c = y_mean - m * x_mean$$

```
print(c)
```

```
...
```

Output:

```
```text
```

0.0

```
...
```

```

```

### # 8. Prediction Formula

Formula:

```
```text
```

$$\hat{y} = mx + c$$

...

Using the previous model:

```
```text
```

$m = 0.2$

$c = 0$

...

If:

```
```text
```

$x = 2500$

...

Then:

```
```text
```

$\hat{y} = 0.2(2500) + 0$

$\hat{y} = 500$

...

Prediction = 500

Python:

```
```python
```

$m = 0.2$

$c = 0$

$x = 2500$

$\text{prediction} = m * x + c$

`print(prediction)`

...

Output:

```
```text
```

500.0

...

---

## # 9. Residual / Error

Residual is the difference between the actual and predicted values.

Formula:

```
```text
```

Residual = $y - \hat{y}$

...

Example:

```text

Actual = 100

Predicted = 90

...

Therefore:

```text

Residual = 100 - 90

= 10

...

Python:

```python

actual = 100

predicted = 90

residual = actual - predicted

print(residual)

...

Output:

```text

10

...

Important:

```text

Positive residual

→ Actual > Predicted

Negative residual

→ Actual < Predicted

...

---

## # 10. Cost Function

Cost Function measures how far predictions are from actual values.

The goal of Linear Regression is:

```text

Minimize Cost

...

A common cost function is MSE.

Formula:

```text

$$J(m,c) = (1/n) \sum (y_i - \hat{y}_i)^2$$

...

Example:

```text

Actual:

[10, 20]

Predicted:

[12, 18]

...

Errors:

```text

$$10 - 12 = -2$$

$$20 - 18 = 2$$

...

Squared errors:

```text

$$4$$

$$4$$

...

Therefore:

```text

$$\text{MSE} = (4 + 4) / 2$$

$$= 4$$

...

$$\text{Cost} = 4$$

---

## # 11. MAE — Mean Absolute Error

MAE measures the average absolute error.

Formula:

```
```text
```

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

```
```
```

Example:

```
```text
```

Actual:

[10, 20]

Predicted:

[12, 18]

```
```
```

Absolute errors:

```
```text
```

$$|10 - 12| = 2$$

$$|20 - 18| = 2$$

```
```
```

Therefore:

```
```text
```

$$\text{MAE} = (2 + 2) / 2$$

$$= 2$$

```
```
```

$$\text{MAE} = 2$$

Python:

```
```python
```

```
from sklearn.metrics import mean_absolute_error
```

```
y_true = [10, 20]
```

```
y_pred = [12, 18]
```

```
mae = mean_absolute_error(y_true, y_pred)
```

```
print(mae)
```

```
```
```

Output:

```text

2.0

...

Interpretation:

```text

MAE = 0

→ Perfect prediction

Lower MAE

→ Smaller average absolute error

...

---

# 12. MSE — Mean Squared Error

Formula:

```text

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

...

Example:

```text

Actual:

[10, 20]

Predicted:

[12, 18]

...

Errors:

```text

[-2, 2]

...

Squared errors:

```text

[4, 4]

...

MSE:

```text

$$\text{MSE} = (4 + 4) / 2$$

$$= 4$$

...

Python:

```
```python
from sklearn.metrics import mean_squared_error
y_true = [10, 20]
y_pred = [12, 18]
mse = mean_squared_error(y_true, y_pred)
print(mse)
```
```

Output:

```
```text
4.0
```
```

Important:

```
```text
MSE penalizes large errors more heavily
because the errors are squared.
```
```

13. RMSE — Root Mean Squared Error

RMSE is the square root of MSE.

Formula:

```
```text
RMSE = $\sqrt{\text{MSE}}$
```
```

Example:

```
```text
MSE = 4
RMSE = $\sqrt{4}$
= 2
```
```

Python:

```
```python
import math
mse = 4
rmse = math.sqrt(mse)
print(rmse)
```
```

Output:

```
```text
2.0
```
```

Advantage:

```
```text
RMSE is expressed in the same unit as the target.
```
```

14. R^2 Score — Coefficient of Determination

R^2 measures how much variation in the target is explained by the model relative to a mean-baseline model.

Formula:

```
```text

$$R^2 = 1 - [\sum(y_i - \hat{y}_i)^2 / \sum(y_i - \bar{y})^2]$$

```
```

Example:

```
```text
```

Actual:

[10, 20, 30]

Predicted:

[12, 18, 28]

```
```
```

Mean:

```
```text
```

$\bar{y} = (10 + 20 + 30) / 3$

$$= 20$$

...

SSE:

```text

$$(10-12)^2 + (20-18)^2 + (30-28)^2$$

$$= 4 + 4 + 4$$

$$= 12$$

...

SST:

```text

$$(10-20)^2 + (20-20)^2 + (30-20)^2$$

$$= 100 + 0 + 100$$

$$= 200$$

...

Therefore:

```text

$$R^2 = 1 - (12 / 200)$$

$$= 1 - 0.06$$

$$= 0.94$$

...

$$R^2 = 0.94$$

Interpretation:

```text

$$R^2 = 1$$

→ Perfect fit

$$R^2 = 0$$

→ Same performance as predicting the mean  
on that evaluation set

$$R^2 < 0$$

→ Worse than predicting the mean

...

Important:

```text

R^2 can be negative.

...

Python:

```
```python
from sklearn.metrics import r2_score
y_true = [10, 20, 30]
y_pred = [12, 18, 28]
r2 = r2_score(y_true, y_pred)
print(r2)
```
```

Output:

```
```text
0.94
```
```

15. Train-Test Split

We divide data into:

```
```text
Training Data
Testing Data
```
```

Example:

```
```text
100 samples
80 → Training
20 → Testing
```
```

Python:

```
```python
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
```

```
test_size=0.2,
random_state=42
)
...
```

Why?

Because we want to evaluate how the model performs on **\*\*unseen data\*\***.

---

## # 16. Build Linear Regression Model

Python:

```
```python  
from sklearn.linear_model import LinearRegression  
model = LinearRegression()  
model.fit(X_train, y_train)  
...
```

`fit()` means:

```
```text
```

Learn the relationship between X and Y  
from the training data.

...

Important:

```
```text
```

LinearRegression()

→ Ordinary Least Squares (OLS)

It does not use Gradient Descent by default.

...

17. Prediction

Python:

```
```python  
y_pred = model.predict(X_test)
...
```

Example:

```
```text
```

Actual:

[200, 300, 400]

Predicted:

[210, 290, 390]

...

The model produces predictions for unseen X values.

18. Coefficients

A coefficient represents the change in the predicted target associated with a 1-unit increase in a feature, **holding the other features constant**.

Python:

```
```python
print(model.coef_)
```
```

Example:

```
```text
Coefficient = 0.2
```
```

Interpretation in a one-feature model:

```
```text
If X increases by 1 unit,
predicted Y increases by approximately 0.2 units.
```
```

In multiple regression:

```
```text
The interpretation is made while holding
the other features constant.
```
```

19. Intercept in Scikit-Learn

Python:

```
```python
print(model.intercept_)
```
```

...

Example:

```text

Intercept = 5

...

Equation:

```text

$$\hat{y} = 2x + 5$$

...

The intercept is the predicted target when all input features are zero.

20. Multiple Linear Regression

Used when there are multiple features.

Formula:

```text

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

...

Example:

```text

House Price =

5000

+ 2000(Age)

+ 300(Size)

...

Given:

```text

Age = 10

Size = 1000

...

Calculation:

```text

$$\text{Price} = 5000 + 2000(10) + 300(1000)$$

$$= 5000 + 20000 + 300000$$

= 325000

...

Prediction:

```text

325000

...

Python:

```python

from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)

prediction = model.predict(X_test)

...

Intermediate Level

21. Vector Form

Multiple Linear Regression can be represented as:

```text

$Y = X\beta + \epsilon$

...

Where:

```text

Y = Target vector

X = Feature matrix

β = Coefficient vector

ϵ = Error vector

...

Example:

```text

Y =

[200

300

400]

...

Feature matrix including an intercept column:

```text

$X =$

[1000 1

1500 1

2000 1]

...

Coefficient vector:

```text

$\beta =$

[0.2

0]

...

Then:

```text

$1000(0.2) + 1(0) = 200$

$1500(0.2) + 1(0) = 300$

$2000(0.2) + 1(0) = 400$

...

Therefore:

```text

$Y = X\beta$

...

---

## # 22. Normal Equation

Normal Equation directly calculates the coefficients for Ordinary Least Squares.

Formula:

```text

$\beta = (X^T X)^{-1} X^T y$

...

Here X normally includes a column for the intercept.

Example:

```text

X =

[1 1

1 2

1 3]

...

```text

y =

[2

4

6]

...

Result:

```text

$\beta$  =

[0

2]

...

Therefore:

```text

$y = 0 + 2x$

$y = 2x$

...

Advantages:

- No learning rate
- No iterative updates

Disadvantages:

- Computational cost can become high with many features
- Direct matrix inversion is not numerically ideal; practical implementations often use more stable linear algebra methods

Important:

```text

$(X^T X)^{-1}$

...

requires the matrix to be invertible. If it is singular, a pseudoinverse or another numerical method can be used.

---

### # 23. Gradient Descent

Gradient Descent is an iterative optimization algorithm.

Goal:

```text

Minimize Cost Function

...

For parameters m and c:

```text

$$m = m - \alpha(\partial J / \partial m)$$

$$c = c - \alpha(\partial J / \partial c)$$

...

Where:

```text

α = Learning Rate

...

Example:

```text

Current m = 1

Gradient = 4

Learning Rate = 0.1

...

Then:

```text

$$\text{New m} = 1 - 0.1(4)$$

$$= 0.6$$

...

The parameter moves in the direction that reduces the cost.

24. Learning Rate

Learning Rate controls the size of each update.

Example:

```
```text
```

Learning Rate = 0.01

```
```
```

Small update.

Large learning rate:

```
```text
```

0.9

```
```
```

May overshoot the minimum or make optimization unstable.

Very small learning rate:

```
```text
```

0.000001

```
```
```

May converge very slowly.

25. Partial Derivative

MSE:

```
```text
```

$$J(m,c) = (1/n)\sum(y_i - \hat{y}_i)^2$$

```
```
```

Since:

```
```text
```

$$\hat{y}_i = mx_i + c$$

```
```
```

Derivative with respect to m:

```
```text
```

$$\partial J / \partial m = -(2/n)\sum x_i(y_i - \hat{y}_i)$$

```
```
```

Derivative with respect to c:

```
```text
```

$$\partial J / \partial c = -(2/n)\sum(y_i - \hat{y}_i)$$

...

Why  $x^2$ ?

Because:

```text

$$d(x^2)/dx = 2x$$

...

Why negative sign?

Because:

```text

$$d(y - mx - c)/dm = -x$$

...

---

## # 26. Feature Scaling

Feature Scaling puts features into comparable numerical ranges.

Two common methods:

```text

Standardization

Min-Max Scaling

...

Example:

```text

Age = 25

Salary = 500000

...

The scales are very different.

Scaling is especially useful for optimization-based models such as:

```text

SGDRegressor

PassiveAggressiveRegressor

...

and is generally important for regularized models such as:

```text

Ridge

Lasso

ElasticNet

...

Tree-based models usually do not require feature scaling.

---

## # 27. Standardization

Formula:

```text

$$z = (x - \mu) / \sigma$$

...

Where:

```text

$\mu$  = Mean

$\sigma$  = Standard Deviation

...

Python:

```python

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

...

Important:

```text

```
fit_transform()
```

→ Training data

```
transform()
```

→ Test data

...

Never fit the scaler separately on the test set.

---

## # 28. Min-Max Scaling

Formula:

```
```text
```

$$x' = (x - \min(x)) / (\max(x) - \min(x))$$

```
```
```

Example:

```
```text
```

$$x = 50$$

$$\min = 0$$

$$\max = 100$$

```
```
```

Therefore:

```
```text
```

$$x' = (50 - 0) / (100 - 0)$$

$$= 0.5$$

```
```
```

Python:

```
```python
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
```
```

Important:

```
```text
```

Fit only on training data.

```
```
```

```

```

## # 29. Polynomial Regression

Polynomial Regression is useful when the relationship between variables is nonlinear.

Formula:

```
```text
```

$$y = b_0 + b_1x + b_2x^2 + \dots + b_nx^n$$

```
```
```

Example:

```
```text
```

$$y = 2 + 3x + x^2$$

```
```
```

If:

```
```text
```

$$x = 2$$

```
```
```

Then:

```
```text
```

$$y = 2 + 3(2) + 2^2$$

$$= 2 + 6 + 4$$

$$= 12$$

```
```
```

Python:

```
```python
```

```
from sklearn.preprocessing import PolynomialFeatures
```

```
from sklearn.linear_model import LinearRegression
```

```
poly = PolynomialFeatures(degree=2)
```

```
X_poly = poly.fit_transform(X)
```

```
model = LinearRegression()
```

```
model.fit(X_poly, y)
```

```
```
```

Important:

```
```text
```

Polynomial Regression is still a linear model

with respect to its coefficients.

```
```
```

```

```

### # 30. Underfitting

Underfitting occurs when a model is too simple to capture the underlying pattern.

Example:

```
```text
```

Actual relationship = Curve

Model = Straight line

...

Symptoms:

```text

Training Error → High

Testing Error → High

...

Solutions:

- Add useful features
- Increase model complexity appropriately
- Reduce excessive regularization

---

### # 31. Overfitting

Overfitting occurs when a model learns the training data too closely, including noise.

Symptoms:

```text

Training Error → Very Low

Testing Error → High

...

Solutions:

- Regularization
- More training data
- Cross Validation
- Feature selection
- Reduce model complexity

32. Bias and Variance

High Bias:

```text

Underfitting

...

High Variance:

```text

Overfitting

...

Goal:

```text

Balance Bias and Variance

...

Example:

```text

Too Simple

→ High Bias

Too Complex

→ High Variance

...

33. Cross Validation

Cross Validation evaluates a model across multiple train-validation splits.

K-Fold Example:

```text

Dataset

↓

Fold 1

Fold 2

Fold 3

Fold 4

Fold 5

...

Each fold becomes validation data once.

Python:

```python

from sklearn.model_selection import cross_val_score

scores = cross_val_score(

model,

X,

```

y,
cv=5,
scoring="r2"
)
print(scores)
print(scores.mean())
...

```

Example output:

```

```text
[0.82, 0.85, 0.80, 0.84, 0.83]
Mean = 0.828
...

```

Cross-validation helps estimate how the model may generalize, but it does not replace a final untouched test set when one is needed.

---

#### # 34. Regularization

Regularization adds a penalty to the loss function to control model complexity.

Main types:

```

```text
Ridge
→ L2
Lasso
→ L1
ElasticNet
→ L1 + L2
...

```

Regularization is commonly used to reduce overfitting and control large coefficients.

35. Ridge Regression

Ridge uses L2 regularization.

Conceptually:

```

```text
Loss = MSE + $\lambda \sum \beta^2$

```

...

Ridge penalizes large coefficients and usually shrinks them toward zero.

Example:

```text

Original coefficient:

$$\beta = 10$$

Squared penalty term:

$$10^2 = 100$$

...

Python:

```python

```
from sklearn.linear_model import Ridge
```

```
model = Ridge(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

...

`alpha` controls regularization strength.

Interpretation:

```text

Higher alpha

→ Stronger regularization

Lower alpha

→ Weaker regularization

...

36. Lasso Regression

Lasso uses L1 regularization.

Formula:

```text

$$\text{Loss} = \text{MSE} + \lambda \sum |\beta|$$

...

Example:

```text

$$\beta_1 = 10$$

$$\beta_2 = 0.2$$

$$\beta_3 = 0$$

...

Lasso can make some coefficients exactly zero.

Therefore Lasso can perform:

```
```text
```

Feature Selection

...

Python:

```
```python
```

```
from sklearn.linear_model import Lasso
```

```
model = Lasso(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

```
...
```

```
---
```

37. ElasticNet

ElasticNet combines L1 and L2 regularization.

Conceptually:

```
```text
```

Loss =

MSE

+ L1 penalty

+ L2 penalty

...

Python:

```
```python
```

```
from sklearn.linear_model import ElasticNet
```

```
model = ElasticNet(
```

```
alpha=1.0,
```

```
l1_ratio=0.5
```

```
)
```

```
model.fit(X_train, y_train)
```

```
...
```

`l1\_ratio` controls the mixture between L1 and L2.

Conceptually:

```
```text
```

$l1\_ratio = 1$

→ L1 component only

$l1\_ratio = 0$

→ L2 component only

```
```
```

38. Ridge vs Lasso vs ElasticNet

```
```text
```

Ridge

↓

L2

↓

Shrinks coefficients

Lasso

↓

L1

↓

Can make coefficients exactly 0

ElasticNet

↓

L1 + L2

↓

Combines both approaches

```
```
```

39. Hyperparameters

Hyperparameters are values selected before or during the model-selection process and are not learned as ordinary model coefficients from each training fit.

Examples:

```
```text
```

Ridge:

alpha

Lasso:

alpha

ElasticNet:

alpha

l1\_ratio

...

Example:

```
```python
```

```
model = Ridge(alpha=10)
```

```
```
```

Here:

```
```text
```

```
alpha = 10
```

```
```
```

is a hyperparameter.

---

#### # 40. Grid Search

GridSearchCV searches through specified hyperparameter combinations using cross-validation.

Example:

```
```python
```

```
from sklearn.model_selection import GridSearchCV
```

```
from sklearn.linear_model import Ridge
```

```
params = {
```

```
"alpha": [0.01, 0.1, 1, 10, 100]
```

```
}
```

```
grid = GridSearchCV(
```

```
Ridge(),
```

```
params,
```

```
cv=5,
```

```
scoring="r2"
```

```
)  
grid.fit(X_train, y_train)  
print(grid.best_params_)  
...
```

Example output:

```
```text  
{'alpha': 10}
...
```

Important:

```
```text  
The selected parameter depends on the data,  
CV strategy, and scoring metric.  
...
```

41. Randomized Search

RandomizedSearchCV samples a specified number of hyperparameter combinations instead of evaluating every possible combination.

Useful when there are many hyperparameters or very large search spaces.

```
```python  
from sklearn.model_selection import RandomizedSearchCV
from sklearn.linear_model import Ridge
params = {
 "alpha": [0.01, 0.1, 1, 10, 100]
}
search = RandomizedSearchCV(
 Ridge(),
 params,
 n_iter=5,
 cv=5,
 scoring="r2",
 random_state=42
)
search.fit(X_train, y_train)
```

...

---

## # Advanced Level

### # 42. Bayesian Ridge

BayesianRidge applies Bayesian principles to linear regression.

It treats model parameters probabilistically and estimates posterior distributions.

Python:

```
```python
from sklearn.linear_model import BayesianRidge
model = BayesianRidge()
model.fit(X_train, y_train)
...

```

Useful for:

```
```text
Regularized linear modeling
Small/medium datasets
Situations where uncertainty estimates are useful
...

```

---

### # 43. Huber Regressor

HuberRegressor is a robust regression model.

It is less sensitive to outliers than ordinary least-squares regression because it uses a loss function that behaves differently for large residuals.

Example:

```
```text
Data:
10
20
30
40
1000
...

```

`1000` is an extreme observation.

Ordinary Linear Regression can be strongly affected by it.

Huber Regression reduces the influence of large residuals.

Python:

```
```python
from sklearn.linear_model import HuberRegressor
model = HuberRegressor()
model.fit(X_train, y_train)
...

```

#### # 44. RANSAC Regressor

RANSAC means:

```
```text
Random Sample Consensus
...

```

It repeatedly fits a model to subsets of data and tries to find a model supported by a large group of observations.

Observations that do not fit the consensus model can be treated as outliers.

Example:

```
```text
Normal points:
10
20
30
40
Outlier:
500
...

```

RANSAC can identify observations that are inconsistent with the main model.

Python:

```
```python
from sklearn.linear_model import RANSACRegressor
model = RANSACRegressor()
model.fit(X_train, y_train)

```

...

45. Theil-Sen Regressor

Theil-Sen is a robust regression method.

It is less sensitive to individual extreme observations than ordinary least squares.

Python:

```
```python
from sklearn.linear_model import TheilSenRegressor
model = TheilSenRegressor()
model.fit(X_train, y_train)
...

```

Useful when:

```
```text
Dataset contains outliers
...

```

46. SGDRegressor

SGDRegressor uses Stochastic Gradient Descent.

It is useful for:

- Large datasets
- Online learning
- Incremental learning

Python:

```
```python
from sklearn.linear_model import SGDRegressor
model = SGDRegressor(
 max_iter=1000,
 random_state=42
)
model.fit(X_train, y_train)
...

```

Concept:

```
```text

```

Traditional Batch Gradient Descent

→ Uses the whole training dataset for each update

Stochastic Gradient Descent

→ Uses individual samples for updates

...

In practice, SGD-based training can use mini-batch-like implementations or sample sequences depending on the algorithm/framework.

Important:

```text

Feature scaling is usually important for SGDRegressor.

...

---

## # 47. PassiveAggressiveRegressor

PassiveAggressiveRegressor is an online learning algorithm.

Basic idea:

```text

Prediction is sufficiently correct

→ Little/no update

Prediction violates the desired loss tolerance

→ Larger update

...

Python:

```python

```
from sklearn.linear_model import PassiveAggressiveRegressor
```

```
model = PassiveAggressiveRegressor(
```

```
 random_state=42
```

```
)
```

```
model.fit(X_train, y_train)
```

...

Useful for:

```text

Streaming data

Online learning

Incremental learning scenarios

...

48. Quantile Regression

Quantile Regression predicts a specific conditional quantile.

Ordinary Linear Regression generally models the conditional mean.

Quantile Regression can model:

```text

10th percentile

50th percentile

90th percentile

...

Example:

```text

quantile = 0.5

...

means median prediction.

Python:

```python

```
from sklearn.linear_model import QuantileRegressor
```

```
model = QuantileRegressor(
```

```
quantile=0.5,
```

```
alpha=0.0
```

```
)
```

```
model.fit(X_train, y_train)
```

...

Example:

```text

Salary Prediction

0.5 quantile

→ Median conditional salary

0.9 quantile

→ Upper conditional quantile

...

49. Tweedie Regressor

TweedieRegressor is a Generalized Linear Model (GLM).

It is useful for certain non-negative target distributions.

Python:

```
```python
from sklearn.linear_model import TweedieRegressor
model = TweedieRegressor(
 power=1.5,
 max_iter=1000
)
model.fit(X_train, y_train)
```
```

Example applications:

```
```text
```

Insurance claims

Claim-related amounts

Other non-negative outcomes

```
```
```

The appropriate `power` depends on the assumed Tweedie distribution and data-generating process.

50. Poisson Regressor

PoissonRegressor is a Generalized Linear Model designed for count-type targets.

Examples:

```
```text
```

Number of customers

Number of calls

Number of accidents

Number of claims

Number of events

```
```
```

Example:

```
```text
```

Hours = 5

Predicted calls = 12

```
...
```

Python:

```
```python
```

```
from sklearn.linear_model import PoissonRegressor
```

```
model = PoissonRegressor()
```

```
model.fit(X_train, y_train)
```

```
...
```

Important:

```
```text
```

Poisson regression is designed for non-negative count-type outcomes and is based on the Poisson distribution.

```
...
```

```

```

## # 51. Gamma Regressor

GammaRegressor is a Generalized Linear Model for positive continuous target values under suitable distributional assumptions.

Examples:

```
```text
```

Insurance claim severity

Medical cost

Transaction amount

Duration

```
...
```

Example:

```
```text
```

Age = 35

Predicted medical cost = 25000

```
...
```

Python:

```
```python
from sklearn.linear_model import GammaRegressor
model = GammaRegressor()
model.fit(X_train, y_train)
```
```

Important:

```
```text
```

Gamma regression is appropriate for positive continuous outcomes,
not zero/negative target values.

```
```
```

```

```

## # 52. Generalized Linear Models — GLM

GLM extends ordinary linear regression by allowing different response distributions and link functions.

Important models:

```
```text
```

PoissonRegressor

GammaRegressor

TweedieRegressor

```
```
```

Examples:

```
```text
```

Count data

→ Poisson

Positive continuous data

→ Gamma

Certain non-negative / compound distributions

→ Tweedie

```
```
```

```

```

## # 53. Linear Regression Assumptions

Important classical assumptions include:

```
```text
```

1. Linearity
2. Independence
3. Homoscedasticity
4. Appropriate residual distribution
5. No severe multicollinearity

...

These assumptions matter particularly for:

```text

Coefficient interpretation

Statistical inference

Confidence intervals

Hypothesis testing

...

For prediction, model performance should also be evaluated empirically on appropriate validation/test data.

---

#### # 54. Linearity

The relationship between predictors and the expected target should be appropriately modeled as linear in the model parameters.

Example:

```text

X increases

Y approximately follows a straight-line relationship

...

If the relationship is strongly curved:

```text

Polynomial features

Transformations

or another model

...

may be appropriate.

---

#### # 55. Independence

Observations/errors should be appropriately independent for classical regression assumptions.

Example:

```text

Student A

Student B

Student C

```

If observations are repeated measurements from the same person, independence may not hold.

Time-series data can also violate independence because nearby observations may be correlated.

---

#### # 56. Homoscedasticity

Homoscedasticity means the residual variance is approximately constant across fitted values.

Good pattern:

```text

Residual spread:

small

medium

small

medium

```

Problem:

```text

small spread → large spread

```

The second pattern is called:

```text

Heteroscedasticity

```

---

## # 57. Normality of Residuals

For classical statistical inference, residuals are often assumed to be approximately normally distributed.

Important:

```text

The target variable itself does not necessarily need to be normally distributed.

...

Residual diagnostics are more relevant.

58. Multicollinearity

Multicollinearity occurs when predictor variables are highly correlated.

Example:

```text

House Size

Number of Rooms

Number of Bedrooms

...

These variables may contain overlapping information.

Problems:

- Coefficients can become unstable
- Standard errors can increase
- Coefficient interpretation becomes difficult

---

## # 59. VIF — Variance Inflation Factor

VIF can be used to diagnose multicollinearity.

General interpretation:

```text

Low VIF

→ Less evidence of problematic multicollinearity

High VIF

→ More evidence of multicollinearity

...

Common Python:

```
```python
from statsmodels.stats.outliers_influence import variance_inflation_factor
for i in range(X.shape[1]):
 print(
 variance_inflation_factor(X.values, i)
)
```
```

There is no single universal cutoff that applies to every dataset.

Values such as:

```
```text
5
10
```
```

are commonly used as rules of thumb, but context matters.

60. Outliers

An outlier is an observation that is unusually far from the general pattern.

Example:

```
```text
Salary:
30000
32000
35000
34000
500000
```
```

`500000` may be an outlier depending on the context.

Possible approaches:

- Investigate the data
- Correct data errors
- Transform variables
- Use robust regression

- Remove observations only when justified

Important:

```text

An outlier is not automatically a bad data point.

...

---

## # 61. Robust Regression

Robust regression methods reduce sensitivity to outliers.

Examples:

```text

HuberRegressor

RANSACRegressor

TheilSenRegressor

...

Concept:

```text

Ordinary Regression

→ Can be strongly affected by extreme observations

Robust Regression

→ Reduces the influence of extreme observations

...

Different robust methods handle outliers in different ways.

---

## # 62. Residual Analysis

Residual:

```text

$$e_i = y_i - \hat{y}_i$$

...

Residual analysis helps identify:

- Non-linearity
- Heteroscedasticity
- Outliers
- Model misspecification

Example:

```text

Actual: 100

Predicted: 90

Residual = 10

...

---

### # 63. Residual Plot

A residual plot commonly shows:

```text

X-axis → Predicted values

Y-axis → Residuals

...

A useful residual pattern is roughly centered around zero without a systematic structure.

Pattern:

```text

Random around 0

→ Generally desirable

...

Pattern:

```text

Curve

→ Possible non-linearity

...

Pattern:

```text

Funnel shape

→ Possible heteroscedasticity

...

---

### # 64. Adjusted $R^2$

Regular  $R^2$  can increase or remain unchanged when additional predictors are added.

Adjusted  $R^2$  accounts for the number of predictors.

Formula:

```text

Adjusted R^2 =

$$1 - [(1 - R^2)(n - 1) / (n - p - 1)]$$

...

Where:

```text

n = Number of observations

p = Number of predictors

...

Example:

```text

$$R^2 = 0.90$$

$$n = 100$$

$$p = 5$$

...

Then:

```text

Adjusted  $R^2$

$$= 1 - [(1 - 0.90)(99)/(94)]$$

$$= 1 - [0.1 \times 99 / 94]$$

$$= 1 - 9.9 / 94$$

$$\approx 1 - 0.105319$$

$$\approx 0.894681$$

...

Therefore:

```text

$$R^2 \approx 0.90$$

$$\text{Adjusted } R^2 \approx 0.895$$

...

65. Confidence Interval

A Confidence Interval represents uncertainty about a population/model parameter.

Example:

```
```text
```

Coefficient = 5.2

95% Confidence Interval:

[4.8, 5.6]

```
```
```

This is about uncertainty in the estimated parameter, not the range of individual future observations.

66. Prediction Interval

Prediction Interval represents uncertainty around an individual future observation.

Example:

```
```text
```

Predicted salary = 50000

Prediction Interval:

[45000, 55000]

```
```
```

Prediction intervals are generally wider than confidence intervals because they include:

```
```text
```

Parameter uncertainty

+

Individual outcome variability

```
```
```

67. Log Transformation

Log transformation can help with strongly skewed variables or multiplicative relationships.

Example:

```
```python
```

```
import numpy as np
```

```
y_log = np.log1p(y)
```

```
```
```

After prediction:

```
```python
y_pred_original = np.exp(y_pred_log)
```
```

Example:

```
```text
```

Original:

[10, 100, 1000, 10000]

Log transformed:

Much more compressed scale

```
```
```

Important:

```
```text
```

When transforming the target, evaluate predictions appropriately after inverse transformation if the metric is intended to be interpreted on the original scale.

```
```
```

```
---
```

68. Data Leakage

Data Leakage happens when information that should not be available during prediction is accidentally used during training.

Bad example:

```
```text
```

Use future information

↓

Train model

↓

Excellent test score

```
```
```

But the model will not have that future information in real use.

Another common example:

```
```text
```

Fit scaler on the entire dataset

before train-test split

...

This allows information from the test set to influence preprocessing.

Correct:

```
```python
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```
```

Better approach:

```
```text
```

Use Pipeline

...

for preprocessing and model training.

69. Pipeline

Pipeline combines preprocessing and model training.

Example:

```
```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
pipeline = Pipeline([
 ("scaler", StandardScaler()),
 ("model", Ridge(alpha=1.0))
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```
```

Advantages:

- Prevents preprocessing mistakes
- Helps prevent data leakage
- Makes workflow cleaner
- Works well with cross-validation

- Works well with hyperparameter tuning

70. Model Evaluation

Python:

```
```python
from sklearn.metrics import (
 mean_absolute_error,
 mean_squared_error,
 r2_score
)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2:", r2)
```
```

Example:

```
```text
MAE = 10
MSE = 150
RMSE = 12.25
R² = 0.90
```
```

Remember:

```
```text
```

For MAE/MSE/RMSE:

Lower is generally better.

For R²:

Higher is generally better for the same evaluation setup.

```
```
```

71. MAE vs MSE vs RMSE vs R²

```text

MAE

- Average absolute error
- Easy to interpret
- Less sensitive to outliers than MSE

MSE

- Average squared error
- Strongly penalizes large errors

RMSE

- Square root of MSE
- Same unit as target

R<sup>2</sup>

- Measures performance relative to a mean-baseline
- Scale-free

```

72. MAPE

Formula:

```text

$$\text{MAPE} = (100/n) \sum |(y_i - \hat{y}_i) / y_i|$$

```

Example:

```text

Actual = 100

Predicted = 90

```

Percentage Error:

```text

$$|100 - 90| / 100 \times 100$$

= 10%

```

Important:

```
```text
```

MAPE is undefined when actual = 0.

It can also behave poorly when actual values are very close to zero.

```
...
```

```

```

### # 73. MSLE

MSLE = Mean Squared Logarithmic Error.

It compares logarithmic differences between actual and predicted values.

Useful when:

```
```text
```

Target is non-negative

Relative differences matter

```
...
```

Python:

```
```python
```

```
from sklearn.metrics import mean_squared_log_error
```

```
msle = mean_squared_log_error(
```

```
y_test,
```

```
y_pred
```

```
)
```

```
...
```

Important:

```
```text
```

MSLE requires non-negative actual and predicted values.

```
...
```

```
---
```

74. Model Selection

Different regression models are suitable for different data characteristics.

```
```text
```

Linear relationship

→ LinearRegression

Overfitting / correlated features

→ Ridge

Feature selection

→ Lasso

Need both L1 and L2

→ ElasticNet

Outliers

→ Huber / RANSAC / TheilSen

Large-scale SGD training

→ SGDRegressor

Online learning

→ PassiveAggressiveRegressor

Conditional quantile

→ QuantileRegressor

Count-type target

→ PoissonRegressor

Positive continuous target

→ GammaRegressor

Certain non-negative distributions

→ TweedieRegressor

...

This is a **use-case mapping**, not a ranking of models.

---

## # 75. Model Comparison

Model	Main Idea
LinearRegression	Ordinary Least Squares
Ridge	L2 Regularization
Lasso	L1 Regularization
ElasticNet	L1 + L2
BayesianRidge	Bayesian linear regression
HuberRegressor	Robust regression
RANSACRegressor	Consensus-based robust regression

SGDRegressor	Stochastic Gradient Descent
PassiveAggressiveRegressor	Online learning
TheilSenRegressor	Robust regression
QuantileRegressor	Conditional quantile prediction
TweedieRegressor	Tweedie GLM
PoissonRegressor	Count-type GLM
GammaRegressor	Positive continuous GLM

---

## # 76. Visualization

Scatter Plot:

```
```python
import matplotlib.pyplot as plt
plt.scatter(y_test, y_pred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.show()
```
```

Interpretation:

```
```text
Actual  $\approx$  Predicted
→ Better agreement
```
```

A useful additional plot is predicted vs actual with a reference line:

```
```text
y = x
```
```

Points close to this line indicate closer agreement between predictions and actual values.

---

## # 77. Saving Model

Using Joblib:

```
```python
import joblib
```

```
joblib.dump(model, "model.pkl")
```

```
...
```

Load:

```
```python
```

```
model = joblib.load("model.pkl")
```

```
...
```

Important:

```
```text
```

The same compatible Python/library environment

should generally be used when loading a serialized model.

```
...
```

```
---
```

78. Complete Regression Project

Step 1:

```
```text
```

Load Dataset

```
...
```

Step 2:

```
```text
```

Understand Dataset

```
...
```

Step 3:

```
```text
```

Data Cleaning

```
...
```

Step 4:

```
```text
```

Handle Missing Values

```
...
```

Step 5:

```
```text
```

EDA

```
...
```

Step 6:

```text

Feature Engineering

...

Step 7:

```text

Train-Test Split

...

Step 8:

```text

Preprocessing

...

Step 9:

```text

Train Model

...

Step 10:

```text

Prediction

...

Step 11:

```text

Evaluation

...

Step 12:

```text

Cross Validation

...

Step 13:

```text

Hyperparameter Tuning

...

Step 14:

```
```text
```

Model Comparison

```
...
```

Step 15:

```
```text
```

Final Model

```
...
```

Step 16:

```
```text
```

Save Model

```
...
```

Step 17:

```
```text
```

Deploy

```
...
```

```

```

# 79. Complete Python Example

Example Dataset:

```
```python
```

```
import numpy as np
```

```
X = np.array([
```

```
[1000],
```

```
[1500],
```

```
[2000],
```

```
[2500],
```

```
[3000]
```

```
])
```

```
y = np.array([
```

```
200,
```

```
300,
```

```
400,
```

```
500,
```

```
600
```

```
)
```

```
...
```

Split:

```
```python
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X,
```

```
y,
```

```
test_size=0.2,
```

```
random_state=42
```

```
)
```

```
...
```

Train:

```
```python
```

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
...
```

Prediction:

```
```python
```

```
y_pred = model.predict(X_test)
```

```
...
```

Evaluation:

```
```python
```

```
from sklearn.metrics import (
```

```
mean_absolute_error,
```

```
mean_squared_error,
```

```
r2_score
```

```
)
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = mse ** 0.5
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2:", r2)
...
```

Coefficient:

```
```python
print("Coefficient:", model.coef_)
...
```

Intercept:

```
```python
print("Intercept:", model.intercept_)
...
```

For this dataset, the underlying relationship is:

```
```text
y = 0.2x
...
```

So the learned coefficient should be approximately:

```
```text
0.2
...
```

and the intercept approximately:

```
```text
0
...
```

Note:

```
```text
```

This dataset is intentionally perfectly linear and very small, so it is useful for learning concepts but not representative of a real-world regression dataset.

```
...
```

```
---
```

80. Interview Important Questions

Beginner

1. What is Regression?
2. What is Linear Regression?
3. What is Simple Linear Regression?
4. What is Multiple Linear Regression?
5. What is the difference between Regression and Classification?
6. What is an independent variable?
7. What is a dependent variable?
8. What is slope?
9. What is intercept?
10. What is residual?
11. What is prediction?
12. What is Cost Function?
13. What is MSE?
14. What is MAE?
15. What is RMSE?
16. What is R^2 ?

Intermediate

17. How is slope calculated?
18. How is intercept calculated?
19. Why do we minimize MSE?
20. Why does MSE square the error?
21. Why is RMSE useful?
22. Can R^2 be negative?
23. What is Multiple Linear Regression?
24. What is Normal Equation?
25. What is Gradient Descent?
26. What is Learning Rate?
27. What happens if learning rate is too large?
28. What happens if learning rate is too small?
29. What is Feature Scaling?
30. Standardization vs Min-Max Scaling?
31. What is Polynomial Regression?

- 32. What is Overfitting?
- 33. What is Underfitting?
- 34. What is Bias?
- 35. What is Variance?
- 36. What is Cross Validation?

Advanced

- 37. What is Regularization?
- 38. Ridge vs Lasso?
- 39. Lasso vs ElasticNet?
- 40. What is L1 Regularization?
- 41. What is L2 Regularization?
- 42. What is Multicollinearity?
- 43. What is VIF?
- 44. What are Linear Regression assumptions?
- 45. What is Homoscedasticity?
- 46. What is Heteroscedasticity?
- 47. What is Residual Analysis?
- 48. What are outliers?
- 49. How do you handle outliers?
- 50. What is HuberRegressor?
- 51. What is RANSACRegressor?
- 52. What is TheilSenRegressor?
- 53. What is BayesianRidge?
- 54. What is SGDRegressor?
- 55. What is PassiveAggressiveRegressor?
- 56. What is QuantileRegressor?
- 57. What is TweedieRegressor?
- 58. What is PoissonRegressor?
- 59. What is GammaRegressor?
- 60. What is Data Leakage?
- 61. What is Pipeline?
- 62. What is Hyperparameter Tuning?
- 63. What is GridSearchCV?

64. What is RandomizedSearchCV?
65. What is Adjusted R^2 ?
66. Confidence Interval vs Prediction Interval?
67. How do you select a regression model?
68. How do you deploy a regression model?
69. How do you interpret coefficients?
70. How do you diagnose a poor regression model?

81. Quick Revision Table

Topic	Remember
-----	-----
Regression	Predict continuous numerical values
Linear Regression	$\hat{y} = mx + c$
Multiple Regression	$y = b_0 + b_1x_1 + \dots + b_nx_n$
Slope	Rate of change
Intercept	Predicted Y when X = 0
Residual	Actual – Predicted
MAE	Average absolute error
MSE	Average squared error
RMSE	$\sqrt{\text{MSE}}$
R^2	Performance relative to mean-baseline
Adjusted R^2	R^2 adjusted for number of predictors
Ridge	L2
Lasso	L1
ElasticNet	L1 + L2
Gradient Descent	Iterative optimization
Normal Equation	Direct OLS coefficient solution
Cross Validation	Multiple validation splits
Huber	Robust to large residuals
RANSAC	Consensus-based robust regression
Theil-Sen	Robust regression
SGD	Stochastic optimization
Passive Aggressive	Online learning

Quantile	Conditional quantile
Poisson	Count-type target
Gamma	Positive continuous target
Tweedie	Certain non-negative distributions
VIF	Multicollinearity diagnostic
Pipeline	Preprocessing + model
GridSearchCV	Hyperparameter search
RandomizedSearchCV	Random hyperparameter search

82. Final Short Summary

Linear Regression:

```text

$$\hat{y} = mx + c$$

...

Multiple Linear Regression:

```text

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

...

Normal Equation:

```text

$$\beta = (X^T X)^{-1} X^T y$$

...

Gradient Descent:

```text

$$\theta = \theta - \alpha \nabla J(\theta)$$

...

Regularization:

```text

Ridge

→ L2

Lasso

→ L1

ElasticNet

→  $L1 + L2$

...

Main Metrics:

```text

MAE

MSE

RMSE

R^2

Adjusted R^2

MAPE

MSLE

...

Robust Regression:

```text

HuberRegressor

RANSACRegressor

TheilSenRegressor

...

Optimization / Online Learning:

```text

SGDRegressor

PassiveAggressiveRegressor

...

GLM:

```text

PoissonRegressor

GammaRegressor

TweedieRegressor

...

Advanced Concepts:

```text

Cross Validation

Regularization

Hyperparameter Tuning

Feature Scaling

Polynomial Regression

Multicollinearity

VIF

Residual Analysis

Outlier Handling

Data Leakage

Pipeline

Model Selection

Model Saving

Model Deployment

...

Complete Flow:

```
```text
```

Dataset

↓

EDA

↓

Cleaning

↓

Feature Engineering

↓

Train-Test Split

↓

Preprocessing

↓

Model Training

↓

Prediction

↓

Evaluation

↓

Cross Validation



Hyperparameter Tuning



Final Model



Save



Deploy

...